

Reflection

Angela Andrea Castro García A01640019

This integrated activity was a good opportunity to apply different string manipulation and algorithm design techniques to a more realistic cybersecurity problem: analyzing intercepted data transmissions to identify possible malicious code. What made the activity interesting was that the goal was not only to find the required patterns, but also to think about how to do it efficiently while following the coding standards we were given.

For **Part 1**, the objective was to find specific malicious code sequences inside larger transmission streams. Instead of using a simple $O(N * M)$ search, we implemented the **Knuth-Morris-Pratt (KMP) algorithm**. Before searching, the algorithm creates an LPS (Longest Proper Prefix which is also a Suffix) array for the pattern, which takes $O(M)$ time and space. This allows the actual search to be done in $O(N + M)$ time, since the algorithm avoids going back through characters that have already been analyzed. This becomes especially useful when working with very large transmission files, since the search can be performed in a linear way.

For **Part 2**, the objective is to identify the longest "mirrored" code, or palindrome, within the transmissions, so we decided to implement Manacher's Algorithm. While a Dynamic Programming approach would have been simpler to implement, it runs in $O(N^2)$ time and space, which scales poorly for large transmission files. Instead, Manacher's Algorithm provides an optimal $O(N)$ time complexity and $O(N)$ space complexity. By pre-processing the string with separator characters (like #), the algorithm elegantly handles both odd and even-length palindromes uniformly. Furthermore, it leverages previously computed palindrome radii (`p_array`) and a dynamic right boundary (`right_limit`) to avoid redundant character comparisons. Implementing this was challenging but resulted in a highly robust and optimal solution for finding the longest mirrored sequence.

For **Part 3**, we looked at the similarity between the two transmission files by finding their Longest Common Substring. For this, we used **Dynamic Programming** and created a 2D matrix to keep track of characters that matched between both transmissions. This allowed us to find the longest sequence of consecutive characters shared by both files. The solution has a time and space complexity of $O(N * M)$, where N and M represent the lengths of the two transmissions. Although this can require a significant amount of memory for very large files, it provides a reliable way to find the required common substring.

Overall, this activity helped me understand that solving a computational problem is not only about finding an algorithm that works. It is also about thinking about how efficiently that algorithm works and how the solution can be written in a clean and understandable way. Balancing time and space complexity with good programming practices made the activity feel more complete, since both the algorithm itself and the way it was implemented were important for creating a good solution. It was so important for us to divide the work fairly but also let everyone in the team understand what was done to gain as much value as possible from this activity.